HW5 Report: Custom Skill + MCP Integration

Part 1: Custom Skill (50%)

The `/fix-issue` Skill

We built a custom Claude Code skill called `/fix-issue` that automates the full development workflow for implementing a GitHub issue: fetch the issue, create a branch, plan the implementation, follow TDD (RED/GREEN/REFACTOR), and open a PR with a structured review template.

Skill file: [`.claude/skills/fix-issue/SKILL.md`]

Usage:

```
/fix-issue 5
/fix-issue 18
```

Key steps in the workflow:

1. **Read the issue** — Fetch from GitHub via MCP, extract acceptance criteria
2. **Create a branch** — `feat/issue-<number>-<short-slug>` from latest `main`
3. **Plan** — Restate criteria, identify files, flag security concerns, list env vars. Stop and wait for user confirmation.
4. **TDD RED** — Write all tests first, commit, confirm they fail
5. **TDD GREEN** — Write minimum code to pass, commit
6. **TDD REFACTOR** — Clean up, lint, format, commit
7. **Coverage check** — Enforce thresholds (70% overall, 90% on critical paths)
8. **Open PR** — Push branch, open PR with C.L.E.A.R. review and AI disclosure

v1 to v2 Iteration

We first tested v1 on **issue #18** (Clerk authentication integration). The skill successfully fetched the issue, created a branch, produced a detailed plan, and followed the TDD cycle. However, several pain points emerged:

- **CI failures due to missing secrets:** The skill never checked whether new environment variables (e.g., `CLERK_SECRET_KEY`) were configured in GitHub Actions. Local tests passed, but CI failed repeatedly.
- **Prettier formatting mismatches:** Every commit risked a CI Prettier failure because v1 only ran `format:check` (which reports errors) instead of `prettier --write` (which fixes them).
- **Wrong branch commits:** The skill said "confirm branch is active" but didn't enforce it. In one run, early test commits landed on a stale branch instead of the intended feature branch.
- **`.env.example` forgotten:** New env vars were added as an afterthought rather than as part of the implementation checklist.

Changes in v2:

Problem in v1	Fix in v2
CI secrets not checked	Added env var pre-flight checklist in the Plan step
Prettier mismatches	Inserted `npx prettier --write .` before every commit
Wrong branch commits	Added explicit `git branch --show-current` assertion before any commit

`.env.example` forgotten	Made it an explicit GREEN-phase checklist item
"Done" = local green only	Clarified that CI must pass on the PR before the issue is considered closed
CI pipeline not updated	Added a new step 7 for updating `ci.yml` when new env vars are introduced

We tested v2 on **issue #5** (job description intake), where the improved workflow ran smoothly without the CI surprises we hit in v1.

Part 2: MCP Integration (35%)

We connected two MCP servers to our Claude Code workflow: **GitHub MCP** and **Context7 MCP**.

GitHub MCP

The GitHub MCP server provides direct access to our repository's issues, PRs, and file contents from within Claude Code sessions.

Setup (`.mcp.json`):

```
"github": {
  "command": "docker",
  "args": ["run", "-i", "--rm", "-e", "GITHUB_PERSONAL_ACCESS_TOKEN",
"ghcr.io/github/github-mcp-server"],
  "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_PERSONAL_ACCESS_TOKEN}" }
}
```

What it enables: The `/fix-issue` skill uses GitHub MCP to fetch issue details (title, acceptance criteria, labels) automatically. Instead of manually copying issue descriptions into the conversation, Claude reads them directly from the repository — ensuring it always works with the latest version of the requirements.

Context7 MCP

Context7 fetches up-to-date library documentation on demand, so Claude Code can query the latest API docs instead of relying on potentially outdated training data.

Setup:

```
claude mcp add context7 -s user -- npx -y @upstash/context7-mcp@latest
```

What it enables: We used Context7 to fix a Prisma deprecation warning. The `driverAdapters` preview feature had been promoted to GA (generally available) in Prisma 7.5.0, so keeping it in `previewFeatures` triggered a deprecation warning. Claude Code used Context7's `resolve-library-id` to find the Prisma docs, then `query-docs` to confirm the feature was now stable. The fix was to remove `previewFeatures = ["driverAdapters"]` from `prisma/schema.prisma` and regenerate the client. Without Context7, Claude would have relied on stale training data and might not have known about this change.

Vercel MCP

The Vercel MCP server connects Claude Code directly to our Vercel deployment environment, enabling it to inspect deployment state, environment variables, and build logs without leaving the coding session.

Setup (`.mcp.json`):

```
"vercel": {
  "command": "npx",
  "args": ["-y", "@robinson_ai_systems/vercel-mcp"],
  "env": { "VERCEL_TOKEN": "${VERCEL_API_TOKEN}" }
}
```

What it enables:

- **Deployment diagnostics** — Claude can check whether a preview deployment succeeded or failed and pull build logs directly, rather than the developer switching to the Vercel dashboard mid-session.
- **Environment variable inspection** — during the CI/CD setup (issue #22), the Vercel MCP let Claude verify which env vars were already set on the project (`vercel_list_env_vars`) before deciding what to add, avoiding duplicate secrets.
- **Preview URL retrieval** — after a PR deploy, Claude can fetch the preview URL and pass it to Playwright MCP for automated smoke testing, enabling a fully in-session deploy-and-verify loop.

Google Stitch MCP

Stitch is Google's UI generation and design system MCP server. It provides access to design projects, screens, and component code directly from within Claude Code.

Setup (`.mcp.json`):

```
"stitch": {
  "type": "http",
  "url": "https://stitch.googleapis.com/mcp",
  "headers": { "X-Goog-Api-Key": "${GOOG_API_KEY}" }
}
```

What it enables:

We used Stitch MCP while building the job description intake UI (issue #5). Instead of writing the UI from scratch or manually inspecting `docs/prototype.html` , Claude called `stitch_list_projects` to find the BypassHire design project, then `stitch_list_screens` to enumerate all screens (Dashboard, New Project Setup, Resume Tailoring, Auto-Fill Preview). It fetched the HTML code for the "New Project Setup" screen, which contained the full design token theme (colors, typography, spacing), component patterns (tab toggles, focus underlines, the tonal layering system), and layout structure. This gave Claude a precise, living reference for the exact color values, font families, and design rules to follow — rather than an approximation from a static file.

The result: Tailwind v4 CSS variables, the sidebar layout, and the intake form were all built to match the prototype's "Digital Architect" design system without any manual design handoff.

Playwright MCP

The Playwright MCP server provides a real browser automation layer inside Claude Code sessions — navigation, clicks, form fills, screenshots, and DOM snapshots.

Setup (`.mcp.json`):

```
"playwright": {
  "command": "npx",
  "args": ["@playwright/mcp@latest"]
}
```

What it enables:

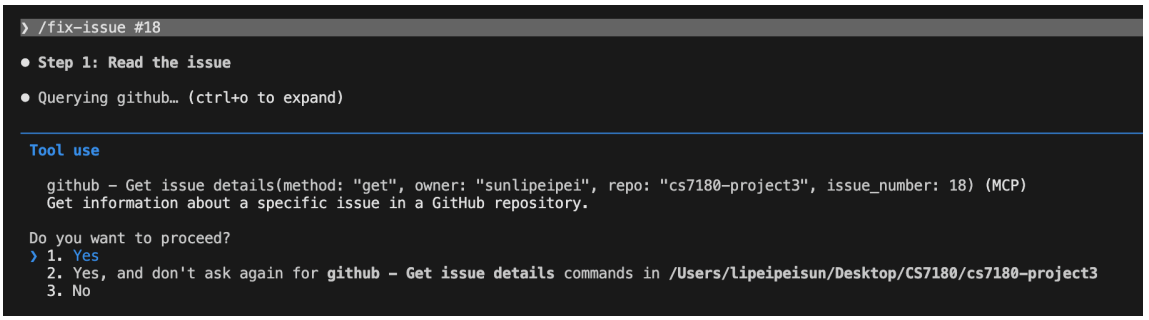
- **Live UI verification** — after deploying locally, Claude can navigate to `http://localhost:3000/dashboard/new`, take a screenshot, and confirm the page renders correctly without the developer switching browser tabs.
- **E2E test scaffolding** — Playwright MCP can interact with the running app and record the exact selectors and interaction sequences, which are then used to write stable Playwright E2E test files.
- **Regression smoke testing** — on every significant UI change, Claude can run a quick browser check against the dev server to catch rendering errors before they reach CI.

During the issue #5 session, we used Playwright MCP to verify the intake form after adding `ClerkProvider` to the root layout — confirming the `MIDDLEWARE_INVOCATION_FAILED` error was resolved and the sign-in page loaded correctly.

Part 3: Screenshots and Session Logs

`/fix-issue` v1 — Issue #18 (Clerk Auth)

The screenshot below shows the skill running on issue #18. Claude fetched the issue via GitHub MCP, created branch `feat/issue-18-clerk-auth`, and produced a detailed plan including a files-to-modify table, security concerns, and risk assessment — then paused for user confirmation before writing code.



▼ Session log: /fix-issue on issue #18 (v1) — click to expand

```
Step 1: Read the issue
  Queried github (ctrl+o to expand)

● Step 2: Create branch

● Bash(git checkout main && git pull origin main && git checkout -b feat/issue-18-
clerk-auth)
  └ Switched to branch 'main'
    Your branch is behind 'origin/main' by 8 commits, and can be fast-forwarded.

● Branch feat/issue-18-clerk-auth is active.
```

```
● Step 3: Plan

  Summary: Integrate Clerk for auth in Next.js 15 App Router — sign-up/sign-in
flows,
  middleware protecting /dashboard and /resume/*, public routes for /, /sign-in,
/sign-up,
  and syncing Clerk's userId to the User table in Neon.

  Acceptance Criteria Checklist
  - Sign-up and sign-in flows work end-to-end
  - /dashboard and /resume/* redirect unauthenticated users to sign-in
  - Public routes (/, /sign-in, /sign-up) are accessible without auth
  - Clerk user ID linked to User record in DB on first sign-in

  Security Concerns
  - CLERK_SECRET_KEY must never be committed — add to .env only
  - Webhook endpoint must verify Svix signature to prevent spoofed events
  - userId for all DB queries must come from auth() server-side

  WAITING FOR CONFIRMATION — proceed with TDD implementation?
```

`/fix-issue` v2 — Issue #5 (JD Intake)

After the v1 iteration, we ran v2 on issue #5. Note the improvements: the branch is verified with `git branch --show-current`, and the plan step explicitly notes that no new env vars or CI changes are needed.

▼ Session log: /fix-issue on issue #5 (v2) — click to expand

```
❯ /fix-issue fix issue #5

● Step 1 — Reading issue #5
  Queried github (ctrl+o to expand)

● Issue #5 — FR-1.1: Accept job description as input (text or URL)
  Accept a job description via pasted text or URL. For URL input, fetch and parse
  the HTML to extract the job posting body. Validate input (non-empty, reasonable
  length) and expose a parseJobDescription function that downstream features can
call.

● Step 2 — Create branch
  Bash(git checkout main && git pull origin main && git checkout -b feat/issue-5-jd-
intake && git branch --show-current)
  └  Branch confirmed: feat/issue-5-jd-intake

● Step 3 — Plan
  Acceptance criteria checklist:
  - User can paste raw job description text
  - User can provide a URL → fetch + parse HTML → extract job posting body
  - Input validated: non-empty, reasonable length (Zod)
  - Returns a structured JobDescription object usable by the tailoring engine
```

```
  Security concerns:
  - SSRF risk (A10): URL input must be validated against ^https?:// before fetch
  - Input validation: Zod schema enforces minLength(1) and maxLength
  - DB scoping: saveJobDescription takes userId from Clerk auth() — never from
request body

  New env vars: None — no new secrets needed.
  No ci.yml changes required.

  WAITING FOR CONFIRMATION — Proceed with this plan? (yes / modify)
```

GitHub MCP — Fetching Issue Details

These screenshots show GitHub MCP in action during the `/fix-issue` workflow, automatically fetching issue metadata from our repository.

```
❯ let's create a new branch and get Issue #23 done

  Queried github (ctrl+o to expand)

● Bash(git remote -v)
  └ origin     https://github.com/sunlipeipei/cs7180-project3.git (fetch)
    origin     https://github.com/sunlipeipei/cs7180-project3.git (push)

● Querying github… (ctrl+o to expand)

 Tool use

   github - Get issue details(method: "get", owner: "sunlipeipei", repo:
   "cs7180-project3", issue_number: 23) (MCP)
   Get information about a specific issue in a GitHub repository.

 Do you want to proceed?
 ❯ 1. Yes
   2. Yes, and don't ask again for github - Get issue details commands in
      /Users/lipeipeisun/Desktop/CS7180/cs7180-project3
   3. No

 Esc to cancel · Tab to amend
```

```
● github - Open new pull request (MCP)(owner: "sunlipeipei", repo: "cs7180-project3", title: "feat: Clerk authentication —
                                      middleware, sign-in/up, webhook, userRepository (#18)", head: "feat/issue-18-clerk-auth",
                                      base: "main", body: "## Summary\n\nIntegrates Clerk for authentication in the Next.js 15 App
                                      Router. Covers all acceptance criteria from Issue #18.\n\n- `middleware.ts` —
                                      `clerkMiddleware()` protecting `/dashboard`, `/resume/*`, and `/api/*`; public routes: `/`,
                                      `/sign-in`, `/sign-up`, `/api/webhooks/clerk`\n- `app/layout.tsx` — wrapped in
                                      `<ClerkProvider>`\n- `app/sign-in/[[...sign-in]]/page.tsx` — Clerk hosted sign-in UI\n-
                                      `app/sign-up/[[...sign-up]]/page.tsx` — Clerk hosted sign-up UI\n-
                                      `app/api/webhooks/clerk/route.ts` — Svix-verified webhook syncing
                                      `user.created`/`user.updated` events to `User` table in Neon\n- `src/lib/userRepository.ts` —
                                       `upsertUser(clerkId, email)` and `getUserByClerkId(clerkId)`\n-
                                      `src/middleware/routeConfig.ts` — `isPublicRoute` / `isProtectedRoute` helpers (pure
                                      functions, fully unit tested)\n- `.env.example` — documents all required Clerk env vars\n\n##
```

Context7 MCP — Fixing Prisma Deprecation

This screenshot shows a complete Context7 MCP workflow: resolving the Prisma library ID, querying docs for the `driverAdapters` deprecation, then applying the fix to `prisma/schema.prisma`.

```
Claude Code v2.1.92
Opus 4.6 (1M context) with medium effort · Claude Max
~/LocalDocument/200-Project/cs7180-project3

❯ help me figure out how to fix this warning. before you start make sure you have access the lattest major vesion information of Prisma using context 7

"Preview feature "driverAdapters" is deprecated. The functionality can be used without specifying it as a preview feature.Prisma "

● context7 - resolve-library-id (MCP)(libraryName: "Prisma", query: "driverAdapters preview feature deprecated in latest Prisma version")

● context7 - query-docs (MCP)(libraryId: "/prisma/prisma/7.5.0", query: "driverAdapters preview feature deprecated, how to remove it from schema")

❯ discuss with me first

● Got it — let me share what I've found and we can discuss before touching anything.

  The warning: Prisma made driverAdapters a stable feature (no longer a preview). So having it in previewFeatures triggers that deprecation warning.

  The fix is simple: remove "driverAdapters" from the previewFeatures array in your Prisma schema. The driver adapter functionality will continue to work
  without it.

  Let me check what your schema currently looks like so we can see the exact change needed.

  Searched for 1 pattern (ctrl+o to expand)

● Here's what we're looking at in prisma/schema.prisma:6:

  generator client {
    provider        = "prisma-client-js"
    output          = "../src/generated/prisma"
    previewFeatures = ["driverAdapters"]   // ← this line
  }

  Since "driverAdapters" is the only preview feature listed, the fix would be to remove the entire previewFeatures line. Your Neon driver adapter setup in
  src/lib/ will keep working — the feature is now GA (generally available), so it doesn't need the flag anymore.

  After removing it, we'd run npx prisma generate to regenerate the client.

  Want me to go ahead with that?
  └ Stop says: Tests failed. Run npm test to see failures and fix before exiting.

● Ran 1 stop hook (ctrl+o to expand)
  └ Tests are failing — fix them before ending the session.

✻ Cooked for 31s

❯ good, make a plan and fix it
```

During this session, Claude also fixed integration tests in `schema.integration.test.ts` that were failing because `DATABASE_URL` was not set locally — adding `describe.skipIf(!hasDb)` guards so tests gracefully skip when no database is available.

Part 4: Retrospective

How did the custom skill change our workflow?

Before `/fix-issue`, implementing a GitHub issue involved a series of manual steps: reading the issue, creating a branch with the right naming convention, planning the implementation, writing tests, checking coverage thresholds, formatting code, and opening a PR with the correct template. Each step was individually simple, but the sequence was easy to get wrong — we would forget to run Prettier before committing, skip the coverage check, or open a PR without linking the issue.

The skill codified this entire workflow into a single command. Claude Code handles the mechanical parts (branch naming, commit message format, PR template, coverage thresholds) while we focus on design decisions that require human judgment — like reviewing the plan before implementation starts. The v1-to-v2 iteration was especially valuable: by running the skill on a real task (issue #18) and observing where it broke, we discovered gaps we would never have anticipated from the skill definition alone. The CI secret pre-flight check in v2 came directly from watching CI fail three times on a feature that passed all local tests.

What did MCP integration enable that wasn't possible before?

MCP servers gave Claude Code access to live, external data sources that it otherwise cannot reach. GitHub MCP eliminated the need to manually copy issue descriptions into the conversation — Claude reads them directly, ensuring it always works with the latest version of the requirements. Context7 MCP solved a

different problem: keeping library knowledge current. When Prisma promoted `driverAdapters` from preview to stable, Claude's training data was stale. Context7 fetched the latest Prisma 7.5.0 docs in real time, confirmed the feature was GA, and Claude applied the correct fix immediately. Without it, we would have had to manually search the Prisma changelog and relay the findings.

The combination of GitHub MCP (project context) and Context7 (library documentation) means Claude Code now operates with both project-specific and ecosystem-wide awareness — two dimensions of context that were previously bottlenecked on the developer manually providing them.

What would we build next?

Three areas stand out:

- **More hooks for automated guardrails.** We already have a PostToolUse hook that runs ESLint after every file edit and a Stop hook that blocks session exit if tests are failing. We would add a pre-commit hook that checks for secrets in staged files and a hook that auto-runs `npx prettier --write` on saved files — removing two more manual steps from the workflow.
- **Sub-agents for parallel work.** Currently, `/fix-issue` runs sequentially: plan, tests, implementation, PR. For larger features, we could spawn sub-agents to handle independent sub-tasks in parallel — one writing unit tests while another scaffolds the API route, then merging their work.
- **A `/deploy-preview` skill** that pushes to a preview branch, triggers a Vercel preview deployment (using the Vercel MCP we already have configured), runs E2E tests against the preview URL with Playwright MCP, and reports the results — turning the deploy-and-verify loop into a single command.